Agentic AI Project Ideas

https://gemini.google.com/app/a670da11c7a470b9

User prompt: Hey Gemini, So I am a 4th year Btech computer science student and i wanted to make a BTech Project with 3 other teammates in this domain : Agentic AI, Harness engineering, efficient Model context window management, Guardrails, MCP Server and client So I want you to first, think and search about the latest technologies that already exists and suggest me new and innovative ideas of Agents that we can build as a group which is really technically strong, maybe integrating modern technologies like Blockchain or other tech and suggest me projects that can be build as well within a month

Response: This is an incredibly ambitious and cutting-edge tech stack for a 4th-year B.Tech project. You are targeting the exact problems the AI industry is currently trying to solve in 2025/2026.

Before diving into the project ideas, here is how your chosen domains currently intersect at the enterprise level:

- **Model Context Protocol (MCP):** Anthropic's open standard acts like a "USB-C cable for AI," standardizing how agents connect to external databases, APIs, and tools.
- **Context Window Management:** Because LLMs have a finite "attention budget," engineers are moving away from stuffing prompts. Instead, they use "Context Compaction" (summarizing history) and structured note-taking to prevent "Context Poisoning"—where hallucinated or outdated information breaks the agent's logic.
- **Harness Engineering:** A concept pioneered at OpenAI, it involves putting AI agents in a strict "harness" of non-functional requirements, static lints, and persona-based LLM-evaluators (Guardrails) that review an agent's work just-in-time before it is executed.

Here are three technically dense, innovative project ideas that integrate your stack, including blockchain, designed to be achievable by a 4-person team in one month.

Idea 1: Immutable DevSecOps "Harness" Agent (The Self-Healing Codebase)

The Concept: An autonomous coding agent that pulls GitHub issues via an MCP server, writes code, and subjects itself to a rigorous "Harness Engineering" pipeline. Every action, test result, and security guardrail approval is cryptographically signed and hashed to a Blockchain testnet for an immutable audit trail.

- **How it works:**
    1. **MCP Client & Server:** You build an MCP server that connects to a local IDE/File System and GitHub.
    2. **Context Window Management:** When the agent tackles a large repository, it uses a "Sub-agent architecture." A Reader Agent summarizes the codebase and provides only the relevant snippets to the Coder Agent.
    3. **Guardrails & Harness:** Before a Pull Request is made, an independent "Reviewer Agent" applies static guardrails (e.g., NeMo Guardrails or strict regex lints) to ensure the code meets team standards.

4. **Blockchain Integration:** To prevent "Shadow AI" (where no human knows why an AI approved malicious code), the Reviewer Agent's cryptographic signature and the specific context it used to make the decision are logged to a smart contract (e.g., on Ethereum Sepolia or Polygon).

- **Why it's great for 1 month:** It limits the scope to a single workflow (Code generation -> Review -> Audit) but goes incredibly deep into the infrastructure.

Idea 2: "Agentic Memory Pager" - A Decentralized Context Server

The Concept: Today, agents lose their minds when context windows overflow. This project builds a standalone MCP Server whose *only job* is to act as an external brain (Long-Term Memory) for *any* LLM client (Claude, Cursor, LangChain).

- **How it works:**

1. **The Core:** You build a memory-management MCP server that uses Vector Databases (like Pinecone/Chroma) to store an agent's past thoughts, actions, and retrieved documents.

2. **Context Compaction Engine:** When the LLM's context window reaches 80% capacity, your system triggers a "compaction" event. It uses a smaller, cheaper LLM to summarize the oldest tokens and stores them in the vector DB, freeing up the main agent's working memory.

3. **Blockchain Integration:** Memory privacy is a huge issue. You can implement decentralized identity (DID). An agent can only access a specific user's memory vault via the MCP server if the user signs a blockchain transaction granting access to their encrypted data.

- **Why it's great for 1 month:** Highly modular. Teammate 1 builds the MCP Server, Teammate 2 builds the Vector/Compaction logic, Teammate 3 handles the Blockchain Auth, and Teammate 4 builds a simple frontend UI to visualize the agent's "brain."

Idea 3: The Smart Contract "Rug-Pull" Auditor Swarm

The Concept: A multi-agent swarm designed to read, simulate, and audit newly deployed Web3 smart contracts in real-time, warning users of vulnerabilities or malicious code before they invest.

- **How it works:**

1. **MCP Integration:** Build a custom Web3 MCP Server that connects to the blockchain (via Ethers.js/Web3.py) to fetch raw bytecode, decompile it, and read on-chain transaction history.

2. **Harness Engineering & Guardrails:** You create a strict "Harness" of known Web3 vulnerabilities (Reentrancy, Integer Overflow). The agents are bound by Guardrails AI to output their findings in a strict, deterministic JSON schema—they are not allowed to hallucinate risk.

3. **Context Management:** Smart contracts can interact with dozens of other contracts. The agent dynamically requests specific contract interfaces into its context window only

when needed, preventing "Context Confusion".

- **Why it's great for 1 month:** It marries Agentic AI directly with Web3 use-cases, providing a highly visual and immediate value proposition (stopping crypto scams) which looks fantastic on a resume and during project presentations.

Suggested 4-Week Execution Plan

To survive a 1-month timeline, you must parallelize immediately:

Week	Technical Milestone	Focus Area
Week 1	**Infrastructure & Prototyping**	Teammates 1 & 2 build the base **MCP Server** (using Python/TypeScript SDKs). Teammates 3 & 4 set up the basic LLM API connections and define the Guardrail schemas.
Week 2	**The Agentic Loop & Memory**	Connect the LLM to the MCP server. Implement the **Context Window Management** logic (deciding when to drop old tokens and summarize data).
Week 3	**Harness & Blockchain Integration**	Implement the secondary "Reviewer" agents to evaluate the primary agent's outputs. Write the Smart Contract for the audit trail.
Week 4	**Testing, UI, & Final Polish**	Stress-test the agents to ensure they don't hallucinate. Build a clean CLI or minimal web dashboard to visualize what the agents are doing behind the scenes.

Sources

1. Top 5 MCP Project Ideas for Seamless AI Integration | by Sebastian Buzdugan | Medium
2. Context Engineering - LLM Memory and Retrieval for AI Agents | Weaviate
3. Harness Engineering: Structuring Context and Guardrails for AI Coding Agents in Production - ZenML LLMOps Database

---

User prompt: See, i do not strictly need this blockchain integration, I just need new and latest technologies similar to that to be used in the project, main focus is on building an agent for a good purpose which is technically relevant. Suggest some more, maybe also a few for college purposes, like solving some issues student face

Response: That is a great pivot. Focusing on a "tech for good" or student-centric problem makes for a much more compelling story during a project defense, and it allows you to test these advanced architectures on messy, real-world data.

By 2026, the industry has formalized **Harness Engineering** as the next evolution after Prompt and Context engineering. While prompt engineering optimizes what you *say* to a model, harness

engineering builds the bounded execution environment (the "harness") that dictates what tools the agent can use, how it handles errors, and when it must stop.

Here are three highly technical, purpose-driven project ideas tailored for the campus environment, utilizing your exact tech stack:

Idea 1: The "Bureaucracy Bypass" Agent (Intelligent Campus Admin Swarm)

The Problem: Students waste countless hours navigating university bureaucracy—figuring out prerequisite waivers, medical leave policies, hostel reimbursement rules, or club funding approvals. The rules are scattered across hundreds of dense, outdated PDFs and portal pages. **The Architecture:**

- **MCP Server (Tooling):** You build an MCP server that securely connects to the university's public portal and a curated Google Drive of student rulebooks.
- **Context Window Management:** University rulebooks exceed standard context limits. You will implement a **Context Compaction** strategy where a smaller model retrieves and summarizes only the exact policy clauses relevant to a student's query before passing it to the main reasoning agent.
- **Guardrails via MCP Gateway:** You deploy an MCP Gateway—which acts as a secure boundary between the agent and the tools. You apply strict OWASP LLM 2025 guardrails. The agent is mathematically constrained: it can *draft* applications and *cite* rules, but a guardrail guarantees it can never hallucinate a policy that doesn't exist or promise a student an approval. **Why it's a winner:** It solves a universal student pain point while demonstrating enterprise-grade document retrieval and strict LLM output bounding.

Idea 2: The Socratic Coding Mentor (Anti-Plagiarism Tutor)

The Problem: When students get stuck on coding assignments, standard AI tools just spit out the final answer. This leads to plagiarism and zero actual learning. **The Architecture:**

- **MCP Client/Server:** The agent connects directly to the student's local IDE (like VS Code) via an MCP server, allowing it to see their active codebase and terminal errors without the student needing to copy-paste.
- **Harness Engineering (Pedagogical Harness):** This is where you shine. You build a "multi-agent control loop". Agent A (the Solver) figures out what is wrong with the student's code. Agent B (the Reviewer/Guardrail) sits in the harness and intercepts Agent A's output. Agent B's sole job is to translate the solution into a *Socratic question* or a hint.
- **Contextual Memory:** The system uses a local vector database to remember what concepts the student struggled with last week, dynamically pulling that context into the current session to tailor its hints. **Why it's a winner:** It flips the narrative on AI in education from "cheating tool" to "personalized teaching assistant," requiring complex agent orchestration to prevent the AI from giving away the answer.

Idea 3: The Accessibility "Bridge" Agent (Real-Time Lecture Context Translator)

The Problem: Differently-abled students (visually impaired, neurodivergent, or non-native speakers) often struggle with inaccessible lecture materials, unformatted PDFs, or professors who speak too fast. **The Architecture:**

- **Live Context Window Management:** The agent processes live lecture transcripts or audio feeds. Because a 2-hour lecture will bloat the context window, your agent uses a "sliding window compaction" technique—continuously summarizing the past 10 minutes into bullet points while keeping the active minute verbatim.
- **MCP Action Servers:** The agent connects to downstream tools to take action on the lecture context. For example, generating accessible alt-text for complex diagrams shown on screen, or structuring messy slide data into clean Markdown.
- **Security & PII Guardrails:** Using the latest MCP security standards, you ensure that any personal data (like a student asking a question in class) is masked before hitting external APIs. **Why it's a winner:** It's an incredibly noble use case that forces your team to handle streaming data, context limits, and real-time tool execution.

How to Split the Work for 4 People (1-Month Plan):

- **Person 1 (The Protocol Engineer):** Builds the **MCP Servers**. They write the APIs that connect the agent to external tools (IDE, PDFs, Portals) using the standard JSON-RPC envelope.
- **Person 2 (The Memory/Context Engineer):** Handles the **Context Window**. They build the logic that decides when to truncate history, summarize old data, and retrieve documents so the agent never crashes from token overload.
- **Person 3 (The Harness/Guardrail Engineer):** Builds the **Execution Loop**. They set up the static guardrails (e.g., using NeMo Guardrails or LlamaGuard) to intercept hallucinations and ensure the agent strictly follows its persona.
- **Person 4 (The Orchestrator & UI):** Wires the LLM (like Claude or GPT) to the harness and builds a clean, minimal chat interface or CLI for students to actually use it.

Sources

1. What Is Harness Engineering for AI Agents? - Milvus
2. What Is an MCP Gateway? The 2026 Definition and Architecture Guide - Future AGI

User prompt: Alright, tell me in short and detailed about this project, What does the modern coding agents like Claude Code do when they compacts and frees their tokens, what will we be doing different and innovate etc : Idea 2: "Agentic Memory Pager" - A Decentralized Context Server The Concept: Today, agents lose their minds when context windows overflow. This project builds a standalone MCP Server whose only job is to act as an external brain (Long-Term Memory) for any LLM client (Claude, Cursor, LangChain). How it works: The Core: You build a memory-management MCP server that uses Vector Databases (like Pinecone/Chroma) to store an agent's past thoughts, actions, and retrieved documents. Context Compaction Engine: When the LLM's context window reaches 80% capacity, your system triggers a "compaction" event. It uses a smaller, cheaper LLM to summarize the oldest tokens and stores them in the vector DB, freeing up

the main agent's working memory. Blockchain Integration: Memory privacy is a huge issue. You can implement decentralized identity (DID). An agent can only access a specific user's memory vault via the MCP server if the user signs a blockchain transaction granting access to their encrypted data. Why it's great for 1 month: Highly modular. Teammate 1 builds the MCP Server, Teammate 2 builds the Vector/Compaction logic, Teammate 3 handles the Blockchain Auth, and Teammate 4 builds a simple frontend UI to visualize the agent's "brain."

Response: Here is a detailed breakdown of how the **"Agentic Memory Pager"** works, why it solves a very real problem that tech giants are actively fighting right now, and how your team will build it.

1. How Modern Agents (Like Claude Code) Handle Memory Today

If you use a modern CLI agent like Claude Code or Cursor, every action—reading a file, writing code, executing a terminal command—consumes "tokens" in the model's context window (which usually maxes out around 200,000 tokens).

When that window hits about 80% to 90% capacity, the agent triggers an **automatic compaction event**.

- It takes the entire conversation history and feeds it back into the LLM, asking for a short summary.
- It deletes the entire long history and replaces it with that tiny summary (compressing, for example, 70,000 tokens down to 4,000).
- **The Result:** The agent remembers the "vibes" and the high-level plan, but completely forgets the granular details—like specific error logs it saw an hour ago, or exactly why a specific architectural decision was made. Unless the user explicitly forced the agent to write a physical `PLAN.md` file to the hard drive, that data is lost forever.

2. The Innovation: What You Will Do Differently

Currently, agents treat context limits like a trash can: once it's full, they compress the top layer and throw the rest away.

Your project introduces **Virtual Memory (Paging)** for AI, inspired by how Operating Systems handle RAM and Hard Drives.

Instead of throwing data away, your **MCP Context Server** acts as an external brain.

- **Pre-Compaction Hooks:** Right before an agent is forced to compact, your server intercepts the data. It forces the agent to extract structured "insights" (decisions, codebase rules, user preferences) and saves them.
- **Persistent Vector Vault:** It pushes the discarded history into a persistent Vector Database (like Chroma or Pinecone).
- **Semantic Paging:** When the agent is working and suddenly needs a detail from 3 hours ago (that was already compacted away), it queries your MCP Server. The server searches the Vector DB, finds the exact historical context, and "pages" it back into the agent's active context window.

3. Architecture Breakdown (How you build it in 1 month)

Since you are dropping the heavy blockchain requirement, we will focus purely on a high-performance, secure MCP integration.

- **Component 1: The MCP Server (The Brain API)**
  - *Tech:* Python or TypeScript using the official Anthropic MCP SDK.
  - *Role:* Exposes tools to the LLM. It gives the agent commands like `store_memory()`, `search_vault()`, and `read_changelog()`.
- **Component 2: The Vector Database (The Hard Drive)**
  - *Tech:* ChromaDB (local) or Pinecone (cloud).
  - *Role:* Converts the agent's thoughts, terminal errors, and file summaries into embeddings (numbers) so they can be searched conceptually later, rather than just by exact keyword.
- **Component 3: The Compaction Engine (The RAM Manager)**
  - *Tech:* A small orchestrator script (using LangChain or simple API calls).
  - *Role:* Monitors the token count. When it hits 80%, it triggers a cheaper model (like Llama 3 or Claude Haiku) to summarize the oldest messages, push the raw logs to the Vector DB, and clear the active window.
- **Component 4: Encrypted Local Identity (The Security Guardrail)**
  - *Tech:* Local AES encryption or AWS KMS.
  - *Role:* Replaces the blockchain idea. The MCP server ensures that Agent A (working on Project X) cannot accidentally pull vector memories from Project Y. Memories are cryptographically tied to the specific user/project workspace.

4. Why this is an A+ B.Tech Project

This project is technically dense and highly relevant to the industry. You aren't just building another "wrapper app" that talks to ChatGPT. You are building **infrastructure for AI**.

You will demonstrate to professors (and future employers) that you understand the fundamental limitations of modern LLMs (context rot, token limits, state amnesia) and that you know how to use the Model Context Protocol (MCP) to engineer a solution around them.

Sources

1. Managing Your Context Window: Clear vs. Compact in Claude Code - Medium

---

User prompt: wait so how will this be integrated into a model, answer in short? Will be be using some open source model and while it is being used for some time and we need to compact the

agent gets triggered?

Response: Here is the short, direct answer to how this actually comes together:

1. How it integrates into the model You do not change the AI model itself. Instead, you use the **Model Context Protocol (MCP)**, which acts like a "USB-C cable" for AI.

- Your LLM runs inside an agent framework (the "Client").
- Your Memory System runs as a standalone app (the "Server").
- They talk to each other via a standardized JSON API. The LLM is given tools it can call, such as `store_memory()`, `search_vault()`, or `create_handoff()`. Whenever the model decides it needs to remember something or look something up, it calls that tool just like it would call a calculator or a weather API.

2. Can you use Open Source Models? Absolutely. In fact, it is highly recommended for this project.

- **The Main Brain:** You can run a powerful open-source model like **Llama-3 (8B or 70B)** or **Qwen 2.5** locally using Ollama or vLLM. (You can also use Claude or GPT-4o if you prefer cloud APIs).
- **The Summarizer (Compaction Model):** This is where open-source shines. You can run a tiny, blazing-fast local model (like Llama-3 8B or Phi-3) whose *only* job is to quickly read old tokens and summarize them for the Vector Database. This makes running the memory system virtually free.

3. How the Compaction gets triggered The trigger doesn't happen inside the LLM; it happens in your **Agent Harness** (the Python/TypeScript code running the loop). Here is the exact flow:

1. **Tracking:** Your code counts every token going in and out of the LLM.
2. **The Threshold:** Once the conversation hits your defined limit (e.g., 80% of the model's max capacity, or simply every 20,000 tokens), your code pauses the main agent.
3. **The Extraction:** Your code takes the oldest part of the chat history (e.g., the first 15,000 tokens) and sends it to the smaller summarizer model.
4. **The Save & Wipe:** The summary is saved into your Vector DB (Chroma/Pinecone). Your code then deletes those 15,000 tokens from the active prompt array, replacing them with a tiny system note (e.g., *"Previous context summarized and saved to memory"*).
5. **Resume:** The main agent resumes its work, completely unaware that it was paused, but now with a fresh, empty context window and the ability to search for its past thoughts if it needs them.

Sources

1. Model Context Protocol (MCP): The Key to Safer, Smarter AI | by Akanksha Sinha - Medium
2. Agent Memory: Shared Context for AI Agents Through MCP - datamcp

---

User prompt: based on this readme explai the tasks for each member in more detaield but simpler terms

Response: Teammate 1: Protocol & Sandbox Engineer (The Storage & API Builder)

- **What you are building in plain English:** You build the "USB port" and data vault for the project. Your job is to create a local server that accepts tool calls from the main AI agent, saves memories into a database, and lets the agent search or fetch old memories whenever it gets confused.
- **Detailed Task Breakdown:**
    1. **Build the FastMCP Server:** Write a local Python server using `FastMCP` that gives the AI 3 core tools:
        - `store_memory`: Saves new summarized context into the database.
        - `search_vault`: Performs vector search to find past memories.
        - `page_context`: Retrieves full context chunks back into the active session.
    2. **Create a Safe Python Sandbox:** Build an isolated Python REPL environment where sub-agents can programmatically run code snippets to inspect or verify stored context objects safely.
    3. **Manage Encryption & Storage:** Set up session keys and local encrypted storage so different projects/sessions keep their memory vaults separate and secure.
- **Your Deliverable:** A running local server (`server/mcp_server.py`) that responds to standardized tool requests from any MCP client.

Teammate 2: Code Compactor / AST Engineer (The Code Shrinker)

- **What you are building in plain English:** You shrink raw code files so they take up 90% less space while keeping all the important structural logic intact.
- **Detailed Task Breakdown:**
    1. **Parse Code Files:** Use syntax parsers like `Tree-sitter` or Python's native `ast` module to read raw source code files and break them into structural components.
    2. **Extract Structural Skeletons:** Build a compression pipeline that strips out inner function logic and heavy implementations, while preserving class names, function signatures, imports, type hints, and docstrings.
    3. **Build Code Graphs:** Generate lightweight dependency maps showing how functions and classes relate to each other, indexing these blueprints into ChromaDB.
- **Your Deliverable:** A pipeline (`compactor/ast_code/code_parser.py`) that compresses a 5,000-token code file into a 500-token structural skeleton without losing functional context.

Teammate 3: Text Compactor / RLM Engineer (The Chat Summarizer)

- **What you are building in plain English:** You shrink long chat histories and conversational logs into tight, structured facts using free local AI models.
- **Detailed Task Breakdown:**
    1. **Chunk Large Conversations:** Write chunking logic that breaks 10,000+ word chat histories into smaller, logical text blocks.
    2. **Set Up Local Sub-LLMs:** Configure parallel local AI models (like `Llama-3-8B` or `Phi-3-Mini` via Ollama) to process these text chunks simultaneously.
    3. **Extract Key Facts (Multi-Pass Compaction):** Program the mini-models to extract key architectural decisions, resolved bugs, and user rules, turning messy conversations into clean, structured fact maps without losing critical meaning.
- **Your Deliverable:** A script (`compactor/rlm_text/recursive_summarizer.py`) that distills 50,000 tokens of chat dialogue into structured fact maps.

Teammate 4: Harness Orchestrator & Guardrails (The Traffic Cop & Manager)

- **What you are building in plain English:** You are the team integrator. You build the main application loop, monitor memory usage, route data to Teammate 2 or 3 when memory gets full, validate outputs, and build a visual dashboard.
- **Detailed Task Breakdown:**
    1. **Build the Central Agent Loop:** Write a state machine using `LangGraph` that controls the main agent's step-by-step thinking loop.
    2. **Implement the 80% Token Trigger:** Create a real-time token monitor. When active memory hits 80% capacity, pause the agent and route code files to Teammate 2's pipeline and conversation text to Teammate 3's pipeline.
    3. **Enforce Strict Guardrails:** Use `Instructor` and `Pydantic` schemas to force all compacted outputs into strict JSON formats before Teammate 1 saves them to the database.
    4. **Build the Observability UI:** Build a simple visual UI using `Streamlit` (or a CLI) that shows live token counts, memory compaction triggers, and active database page-ins.
- **Your Deliverable:** The core execution harness (`harness/agent_loop.py`) and Streamlit dashboard (`ui/app.py`) connecting all three sub-systems together.

User prompt: Fior all of these tasks what would be the appropriate order in which each of these 4 tasks need to be done

Response: To complete the project efficiently within a 1-month timeline, you should execute these tasks in a **4-phase dependency pipeline**.

Phase 1: Foundation & Data Contracts (Week 1 / Days 1–5)

Dependencies: Must happen first so everyone agrees on data formats.

1. **Teammate 1 (Protocol & Storage Setup):** Sets up the ChromaDB/SQLite databases and drafts the base `FastMCP` server structure.
2. **Teammate 4 (Guardrails & Schemas):** Defines the exact `Pydantic` data schemas for what a "compacted memory" looks like in JSON format.

- **Why this order:** Teammates 2 and 3 cannot build summarizers until they know the exact output format (schema) expected by Teammate 1's database.

Phase 2: Core Modules in Parallel (Weeks 1–2 / Days 6–16)

Dependencies: Once Phase 1 sets the contracts, everyone works simultaneously.

- **Teammate 1:** Implements the core MCP tools (`store_memory`, `search_vault`, `page_context`) and builds the isolated REPL sandbox.
- **Teammate 2:** Builds the AST parsing pipeline using `Tree-sitter` or `ast` to compress source code into structural skeletons.
- **Teammate 3:** Sets up local models (`Phi-3` / `Llama-3` via Ollama) and writes the recursive summarization script for dialogue text.
- **Teammate 4:** Builds the base `LangGraph` state machine loop and the Token Tracker module to monitor active context usage.

Phase 3: Integration & Harness Routing (Week 3 / Days 17–23)

Dependencies: Requires Phase 2 individual modules to be functional.

1. **Teammate 4 (Lead Integrator):** Integrates the 80% compaction trigger into the `LangGraph` harness.
2. **Teammate 4 + Teammates 2 & 3:** Wires the 80% trigger so code data routes to Teammate 2's AST parser and conversation text routes to Teammate 3's RLM summarizer.
3. **Teammate 4 + Teammate 1:** Validates the outputs using `Instructor` guardrails and sends the structured memories directly into Teammate 1's MCP server (`store_memory`).

Phase 4: UI, Testing & Polish (Week 4 / Days 24–30)

Dependencies: Requires a connected, working pipeline.

1. **Teammate 4:** Builds the `Streamlit` dashboard to visually display live token usage, compaction events, and database retrieval calls.
2. **Entire Team:** Performs end-to-end testing (running a coding session, filling the context window to trigger compaction, and verifying that the agent successfully retrieves old

memories via `search_vault`).
